

Speaker Script

sTiles: Accelerated Computational Framework
for Sparse Factorizations of Structured Matrices

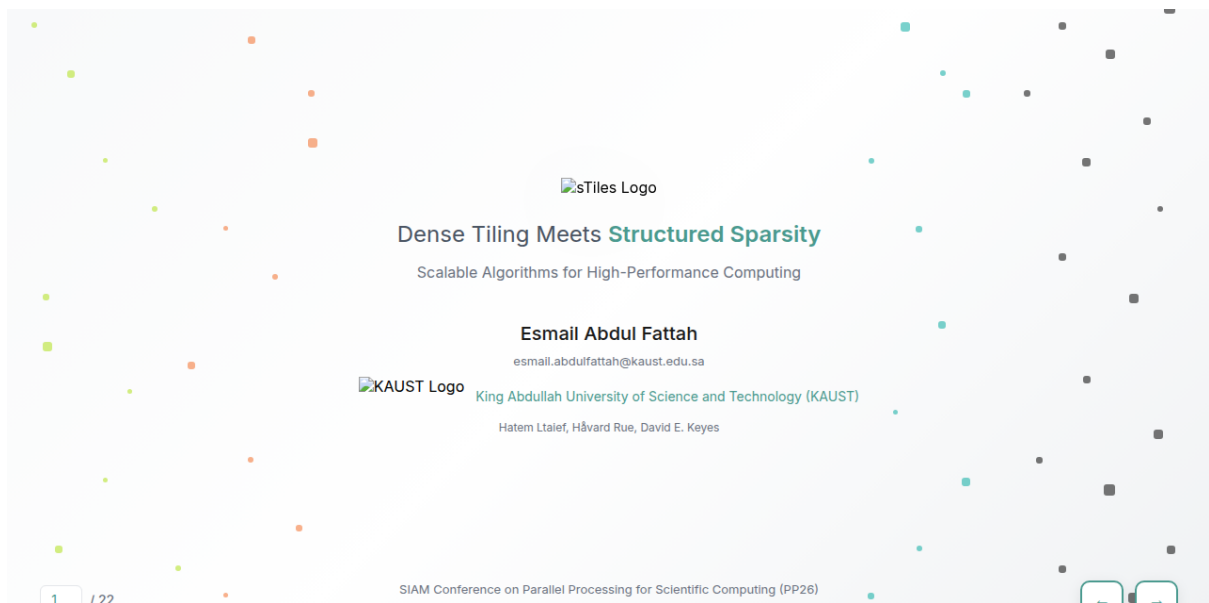
SIAM Conference on Parallel Processing (PP26) — March 2026

Total Presentation Time: 25–30 minutes

Style: Professional, clear, technically precise

Coverage: Two papers (ISC25 + ISC26) plus additional updates

Slide 1: Title Slide



Speaker Notes

Good morning/afternoon everyone. Thank you for attending this session.

Today I'll present sTiles, our GPU-accelerated computational framework for sparse factorizations of structured matrices. This work represents two papers — one on Cholesky factorization published at ISC25, and one on selected inversion targeting ISC26 — plus additional optimizations we've developed since.

The practical motivation is enabling faster Bayesian inference at scale, where these factorizations are the computational bottleneck.

Slide 2: Why This Matters

Why This Matters

| Bayesian Inference at Scale

Major global institutions rely on Bayesian methods that generate **structured sparse matrices** requiring efficient factorization.

[1] WHO. Methods for estimating excess mortality associated with COVID-19 pandemic. *World Health Data Platform*, 2022.

[2] CDC. County-level teen birth rates in the United States. *National Center for Health Statistics*, 2024.

[3] UN DESA. World Population Prospects 2024: Methodology of population estimates. *UN DESA/POP/2024*, 2024.

[4] Fioravanti et al. Interpolating climate variables using INLA and SPDE. *Environmetrics*, 2023.

Speaker Notes

Let me start with why this work matters beyond pure linear algebra.

Major global institutions — the World Health Organization, the CDC, the United Nations — all rely on Bayesian statistical methods for critical analyses. WHO used these techniques to estimate excess COVID-19 mortality. The CDC uses them for epidemiological studies. The UN uses them for World Population Prospects.

All these applications use a methodology called INLA — Integrated Nested Laplace Approximations — which requires **hundreds** of Cholesky factorizations and selected inversions per inference run.

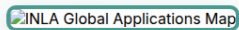
The matrices involved are **structured sparse** matrices, typically arrowhead patterns from Kronecker products of spatial and temporal components. As data resolution increases — more sensors, finer time steps, larger geographic coverage — these matrices grow larger and denser.

This is where sTiles comes in: we need efficient factorization that exploits this structure.

Slide 3: INLA: Global Impact

INLA: Global Impact

INLA enables fast Bayesian inference for latent Gaussian models, powering research in **epidemiology, ecology, climate science, public health,** and beyond.

 INLA Global Applications Map

Source: r-inla.org

3 / 22

Speaker Notes

This map shows the global reach of INLA-based research.

INLA enables fast approximate Bayesian inference for latent Gaussian models. It's become the standard tool in spatial statistics, with applications spanning epidemiology, ecology, climate science, and public health.

The R-INLA package has been used in thousands of published papers across these domains.

Every dot on this map represents research that depends on efficient sparse matrix computations. That's the scale of impact we're targeting with sTiles.

Slide 4: The Computational Challenge

The Computational Challenge

INLA generates **diverse structured sparse matrices** from Kronecker products of spatial, temporal, and other model components.

Dense

Arrowhead

Banded Arrowhead

Sparse Arrowhead

Tridiagonal

Block Diagonal

Block-Tridiagonal

Block-Tridiagonal

Nested Block-Tridiagonal

Block-Diagonal Arrowhead

4 / 22

Speaker Notes

INLA generates diverse structured sparse matrices from Kronecker products of model components — spatial fields, temporal dynamics, fixed effects.

Here you can see ten different sparsity patterns we encounter: dense, arrowhead, banded arrowhead, sparse arrowhead, tridiagonal, block diagonal, block tridiagonal with arrowhead, and various nested block structures.

The challenge is that **neither traditional sparse nor dense solvers are optimal** for these matrices.

[Advance to show solver comparison]

Sparse solvers like CHOLMOD or PARDISO have low arithmetic intensity. They're designed for scattered sparsity, not dense clusters. They suffer from poor cache utilization and irregular memory access.

Dense solvers like PLASMA have $O(n^3)$ complexity. They ignore the sparsity pattern entirely, wasting memory on zeros.

[Advance to show key message]

Neither approach is optimal! We need the efficiency of sparse methods combined with the performance of dense BLAS kernels. That's exactly what sTiles provides.

Slide 5: The sTiles Approach

The sTiles Approach

sTiles = Hybrid framework combining **sparse techniques** (ordering, skip zeros) with **dense performance** (LAPACK/BLAS-3 on tiles)

| Why Tiles?

Inspired by **PLASMA** dense linear algebra library:

- **Uniform grid** of fixed-size dense blocks
- **Regular data structures** for predictable dependencies
- **High arithmetic intensity** via LAPACK/BLAS-3: POTRF, TRSM, SYRK, GEMM
- **Fine-grained parallelism** via tile independence
- **Static scheduling** maximizes data locality

Buttari et al. A class of parallel tiled linear algebra algorithms for multicore architectures. *Parallel Computing*, 2009.

5 / 22

Speaker Notes

sTiles is a hybrid framework combining sparse techniques with dense performance. The key insight comes from PLASMA, the dense linear algebra library: partition the matrix into **uniform fixed-size tiles** and operate on them using highly optimized LAPACK and BLAS-3 kernels — POTRF, TRSM, SYRK, GEMM.

This gives us regular data structures with predictable dependencies, high arithmetic intensity from BLAS-3 operations, fine-grained parallelism through tile independence, and static scheduling that maximizes data locality.

[Advance to show sparse techniques]

But we don't treat everything as dense. We preserve sparsity through preprocessing: permutation using RCM, AMD, or Nested Dissection to minimize fill-in; symbolic factorization to identify which tiles will be nonzero; and most importantly, we **skip zero tiles entirely** during computation.

[Advance to show tile visualization]

Here's the transformation: sparse points become dense tiles. For this banded arrowhead example, we only compute 44 of 100 possible tiles. The gray tiles are never allocated or touched.

This is the essence of sTiles: dense operations where it matters, nothing where it doesn't.

Slide 6: The Fill-in Challenge

The Fill-in Challenge

During Cholesky factorization, **empty tiles can become non-empty** due to GEMM updates

$$\begin{array}{c} \boxed{0} \\ A_{ij} \end{array} = \begin{array}{c} \boxed{0} \\ A_{ij} \end{array} - \begin{array}{c} L_{ik} \\ \text{non-empty} \end{array} \times \begin{array}{c} L_{jk}^T \\ \text{non-empty} \end{array}$$

6 / 22

Speaker Notes

Before we can skip zero tiles, we need to know which tiles will remain zero. This is the fill-in challenge.

During Cholesky factorization, a GEMM update can create fill-in: a tile that starts empty can become non-empty.

Look at this equation: A_{ij} starts as zero. But if both L_{ik} and L_{jk} are non-empty, then the product $L_{ik} \times L_{jk}^T$ is non-zero, and A_{ij} gets filled in.

[Advance to show fill-in result]

This tile is now allocated and must be computed. We can't skip it.

The symbolic factorization phase determines exactly which tiles will have fill-in. Once we know that, we allocate only those tiles and compute only those operations.

For arrowhead matrices with natural ordering, fill-in is **predictable and structured**. The arrowhead shape is preserved in the Cholesky factor, which is why these matrices are particularly amenable to our approach.

Slide 7: The Memory Trade-off

The Memory Trade-off

When we tile, we allocate **full dense storage** for every non-zero tile, even if it's sparse inside!

Sparse Tile (diagonal)



4 elements

Dense Tile (full)



16 elements

7 / 22

Speaker Notes

There's an important trade-off with tile-based approaches: when we tile, we allocate **full dense storage** for every non-zero tile, even if it's sparse inside.

Consider a sparse diagonal tile with only 4 elements on the diagonal. We still allocate storage for all 16 elements in that tile.

[Advance to show allocation]

The sparse tile uses the same memory as a fully dense tile!

Is this worth it? Yes, when the tiles are dense enough. The benefit is that we can use highly optimized dense BLAS kernels without any sparse indexing overhead. For diagonal tiles that are truly sparse, we can use customized kernels or smaller tile sizes.

[Advance to show tile sizes]

The optimal tile size depends on the architecture. For CPUs, we use approximately 120 to fit in L2/L3 cache. For GPUs, we use approximately 600 to maximize occupancy and kernel efficiency.

Slide 8: Ordering Strategies

Ordering Strategies

Permutation is critical: it determines fill-in, parallelism, and whether we preserve structure

Available Orderings

- RCM - Reduces bandwidth
- AMD, COLAMD, CAMD - Minimize fill-in (*SuiteSparse*)
- ND - Graph partitioning (*METIS*)

8 / 22

Speaker Notes

Permutation is critical in sparse factorization. The ordering determines fill-in, parallelism, and whether we preserve the matrix structure.

We support multiple orderings: RCM for bandwidth reduction, AMD and variants from SuiteSparse for fill-in minimization, and Nested Dissection via METIS for graph partitioning.

[Advance to show METIS limitation]

Nested Dissection is attractive because partitions can be processed in parallel. However, METIS isn't always optimal for our matrices. It often creates imbalanced partitions with uneven task distribution, which hurts parallel efficiency.

[Advance to show our approach]

Our approach includes adapted Nested Dissection that balances partition sizes, custom structure-aware orderings, and hybrid combinations.

We can also do tile-level ordering — permuting the tiles themselves after the initial partitioning. And hierarchical approaches: global ND with METIS, then different local orderings per partition.

Slide 9: Ordering Strategies (continued)

Ordering Strategies

Nested Dissection creates independent partitions, but **uniform tiling breaks this independence**



9 / 22

Speaker Notes

Here's an important subtlety: Nested Dissection creates independent partitions at the element level, but uniform tiling can break this independence.

Look at this example. The original sparse matrix after ND has independent blocks. But when we overlay a uniform tile grid, tiles can span partition boundaries. A single tile might contain elements from multiple partitions.

This introduces dependencies between what were supposed to be independent blocks. We have to account for this in our scheduling.

Our adapted ND approach adjusts the separator position to align better with tile boundaries, preserving more of the partition independence at the tile level.

Slide 10: Tree Reduction Explained

Tree Reduction

Exposing parallelism in left-looking Cholesky for arrowhead matrices

The Problem: Sequential Bottleneck

In left-looking Cholesky, many GEMMs update the same target tile sequentially:

 **Sequential GEMMs**

GEMM → GEMM → GEMM → ... (serial dependency)

10 / 22

Speaker Notes

The arrowhead matrix structure creates a parallelism challenge. Look at the DAG for a dense matrix versus an arrowhead matrix. The dense DAG is wide, exposing massive parallelism. The arrowhead DAG is **thin** — there's a long chain of dependent operations. The problem is in the left-looking Cholesky variant: many GEMM operations update the same target tile sequentially. This creates a bottleneck.

Tree reduction solves this. Instead of sequential GEMMs accumulating to one tile, each core computes GEMMs to its own temporary tile in parallel. Then we use a tree-based reduction with GEADD operations to combine the results.

The reduction depth is $\log_2(n)$, so even with thousands of GEMMs, we only need a handful of reduction levels.

The condition: tree reduction helps when the number of accumulations is at least twice the number of available cores. For thick arrowhead matrices with high bandwidth, this is always satisfied.

Slide 11: Algorithm Pipeline

sTiles Pipeline

11 / 22

Speaker Notes

Let me walk through the complete sTiles pipeline.

First, preprocessing: we apply permutation — chosen based on matrix structure — then symbolic factorization to identify nonzero tiles. We allocate only the tiles that will be non-zero.

The numerical factorization uses a customized static scheduler. Each thread has a pre-computed Task Assignment Table — no runtime scheduling overhead. We use a left-looking variant with tree reduction for the accumulation bottleneck.

For GPU execution, each CPU thread manages its own CUDA stream. We replace CPU kernels with cuBLAS and cuSOLVER equivalents: cusolverDnDpotrf, cublasDsyrf, cublasDtrsm, cublasDgemm.

Selected inversion follows the same tile-based approach, computing only the elements of A^{-1} that correspond to the original sparsity pattern.

Slide 12: Test Matrix Characteristics

Test Matrix Characteristics

18 arrowhead matrices from INLA framework covering diverse sizes and structures

PARAMETER	RANGE	MEANING
Matrix Size (n)	10K → 1M	Number of rows/columns
Bandwidth (b)	100 → 15K	Width of banded region
Arrow Thickness (t)	10 → 200	Dense columns at end
Density	0.02% → 4%	Fraction of nonzeros

12 / 22

Speaker Notes

For our experimental evaluation, we use test matrices representative of real INLA applications.

The matrices span three size groups: 10K, 100K, and 500K rows. Within each group, we vary bandwidth from 100 to 3000, and arrowhead thickness of 10 or 200.

Matrix IDs 1 through 18 cover this parameter space. The bandwidth corresponds to the width of the banded diagonal, while arrowhead thickness represents dense blocks at the end — typically fixed effects in statistical models.

For GPU experiments, we add larger matrices: 50K with bandwidth 15,000 to stress-test GPU kernels, and 1 million rows for scalability testing.

These matrices arise from Kronecker products of spatial and temporal precision matrices, exactly what you'd encounter in spatiotemporal statistics.

Slide 13: Performance: Cholesky Factorization

Performance: Cholesky Factorization

Comparison vs state-of-the-art sparse solvers across 18 test matrices (52 cores)

Group 1 Results

13 / 22

Speaker Notes

Now for the results. This slide compares sTiles against state-of-the-art sparse solvers: CHOLMOD, SymPACK, MUMPS, and PARDISO.

The plots show execution time for our test matrices on a 52-core Intel Xeon system. Lower is better.

[Advance to show second plot]

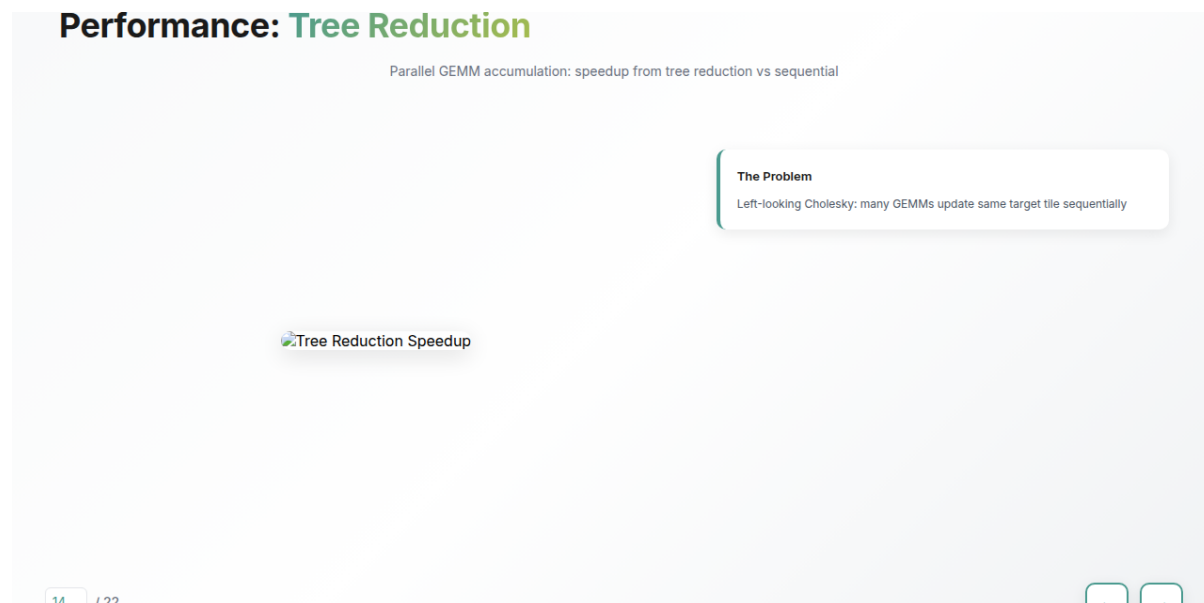
For larger matrices, especially those with higher bandwidth, sTiles dominates.

[Advance to show speedup summary]

The maximum speedups are striking: $11\times$ over PARDISO, $9.3\times$ over SymPACK, $8.4\times$ over CHOLMOD, $5\times$ over MUMPS.

Critically, the performance gap **grows with matrix size**. sTiles scales better because our tile-based approach with static scheduling eliminates the overhead that dynamic approaches accumulate at scale.

Slide 14: Performance: Tree Reduction



Speaker Notes

This plot shows the benefit of tree reduction for parallel GEMM accumulation. The x-axis is core count, y-axis is speedup versus sequential execution. Different curves represent different numbers of GEMMs to accumulate.

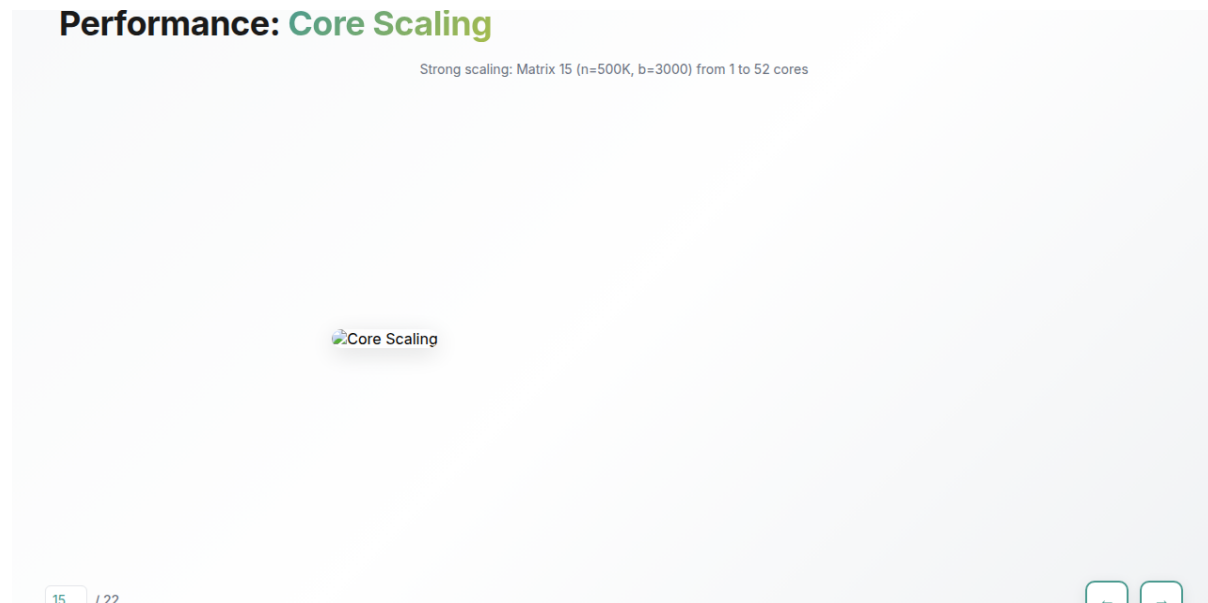
For 50,000 GEMMs, tree reduction achieves up to $5\times$ speedup at 16-32 cores.

[Advance to show explanation]

The problem: left-looking Cholesky has many GEMMs updating the same tile sequentially.

The solution: parallel GEMMs to temporary tiles, then tree-based GEADD accumulation. The sweet spot is when you have enough GEMMs to keep all cores busy during the parallel phase, but not so many that the reduction overhead dominates.

Slide 15: Performance: Core Scaling



Speaker Notes

This is strong scaling: how execution time decreases as we add cores, for a fixed-size matrix.

We use Matrix 15 — 500K rows, bandwidth 3000 — scaling from 1 to 52 cores.

sTiles, shown in lime green, is fastest at all core counts. At 52 cores, it achieves 0.8 seconds versus 12.6 seconds single-threaded — that's $15.6\times$ speedup.

[Advance to show comparisons]

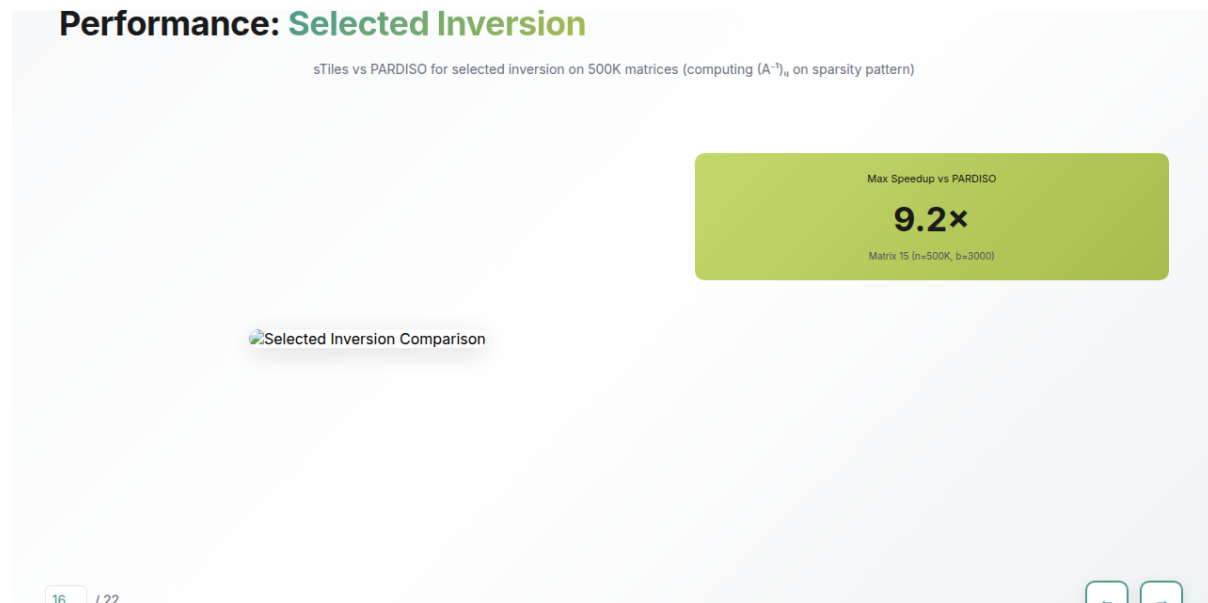
The sparse solvers — CHOLMOD, SymPACK, MUMPS, PARDISO — are $3.8\times$ to $10.7\times$ slower than sTiles at 52 cores.

PLASMA, the dashed line, is the dense baseline. It's $271\times$ slower because it treats the full matrix without exploiting sparsity.

[Advance to show key message]

sTiles exploits structure: sparse storage footprint with dense BLAS kernel performance. Best of both worlds.

Slide 16: Performance: Selected Inversion



Speaker Notes

Now let's look at selected inversion — computing elements of A^{-1} on the sparsity pattern. This operation is essential for Bayesian inference: we need diagonal elements for marginal variances, off-diagonals for covariances.

This plot compares sTiles versus PARDISO for selected inversion on 500K matrices. sTiles achieves up to **9.2×** **speedup** over PARDISO for high-bandwidth matrices.

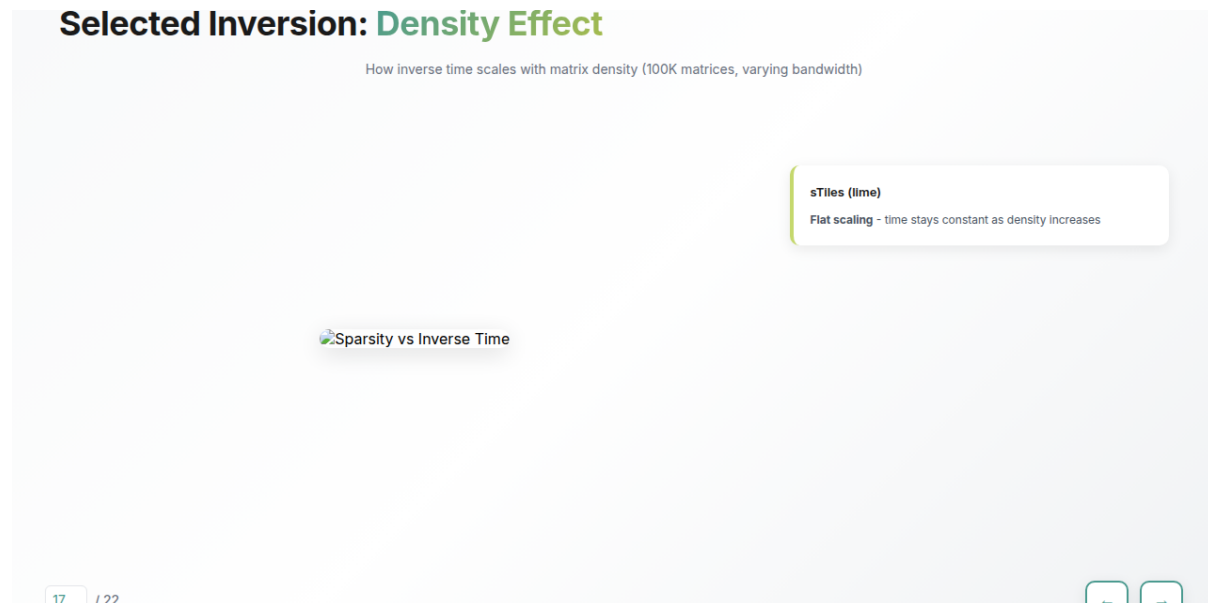
[Advance to show details]

For matrices 14, 15, 17, 18 with higher bandwidth, sTiles is 2.6× to 9.2× faster.

For low-bandwidth matrices like 13 and 16, performance is similar — sparse overhead dominates when there's not enough computational work per tile.

The key insight: our tile-based approach with static scheduling eliminates dynamic overhead that becomes significant at scale.

Slide 17: Selected Inversion: Density Effect



Speaker Notes

This experiment examines how selected inversion time scales with matrix density. We use 100K matrices, progressively increasing the density of the arrowhead band. Two sets: bandwidth 1500 and bandwidth 3000.

sTiles, shown in lime, has **remarkably flat scaling**. Execution time stays nearly constant as density increases.

[Advance to show PARDISO]

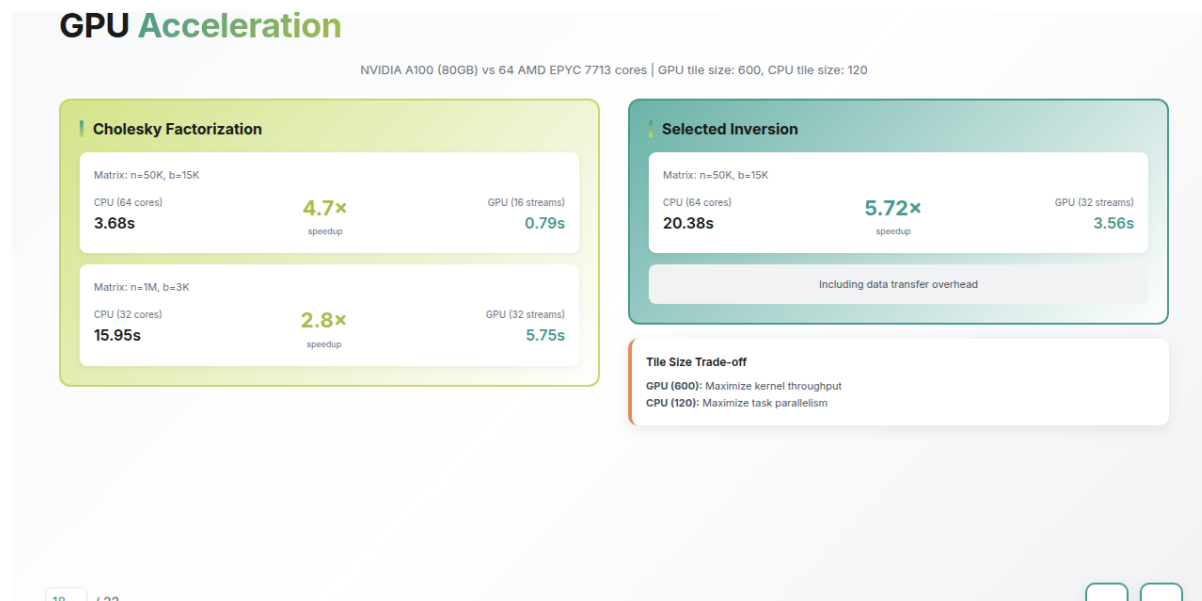
PARDISO, in black, shows growing cost — time increases significantly with density, up to 12× slower than sTiles at high density.

[Advance to show explanation]

Why does sTiles stay flat? Fixed tile size means predictable BLAS workload. Once a tile is allocated, we process it the same way regardless of how full it is.

PARDISO's supernodal approach is more sensitive to the actual nonzero structure within supernodes.

Slide 18: GPU Acceleration



Speaker Notes

Finally, GPU acceleration. We compare an NVIDIA A100 with 80GB memory against a 64-core AMD EPYC CPU.

For Cholesky factorization: the 50K matrix with bandwidth 15K achieves $4.7\times$ GPU speedup — 0.79 seconds on GPU versus 3.68 seconds on 64 CPU cores.

The larger 1M matrix with bandwidth 3K achieves $2.8\times$ speedup. Lower relative speedup because the narrower bandwidth means less computational intensity per tile.

For selected inversion: the 50K matrix achieves $5.7\times$ GPU speedup. Selected inversion has more computation per memory access than factorization, making it even more GPU-friendly.

The GPU implementation uses larger tiles — 600 instead of 120 — to maximize kernel efficiency. Each CPU thread manages its own CUDA stream for concurrent kernel execution.

Slide 19: Dense Matrix Performance



Speaker Notes

For completeness, we also show sTiles performance on fully dense matrices — the limiting case where every tile is nonzero.

Even here, sTiles is competitive with PLASMA, the state-of-the-art dense solver. Both use the same underlying BLAS kernels; sTiles adds minimal overhead from its sparse-aware infrastructure.

This validates that sTiles doesn't sacrifice dense performance to gain sparse efficiency. When the matrix happens to be dense, we match the dense solver performance. When it's sparse, we dramatically outperform sparse solvers.

Slide 20: Key Innovations

Speaker Notes

Let me summarize the key innovations in sTiles.

First, **tile-based hybrid approach**: uniform tiles with dense BLAS kernels, but operating only on nonzero tiles. This bridges sparse and dense linear algebra.

Second, **structure-aware scheduling**: static task assignment eliminates runtime overhead. Each core knows exactly what to compute before numerical work begins.

Third, **tree reduction**: breaks the sequential bottleneck in left-looking Cholesky for arrowhead matrices.

Fourth, **GPU acceleration**: larger tiles for GPU efficiency, CUDA streams for concurrency, cuBLAS/cuSOLVER kernels.

Fifth, **flexible ordering**: supports RCM, AMD, ND, and hybrid combinations, with automatic selection based on matrix structure.

These innovations combine to deliver order-of-magnitude speedups over existing solvers for structured sparse matrices.

Slide 21: Publications and Future Work

Publications & Future Work

Speaker Notes

This work is documented in two papers.

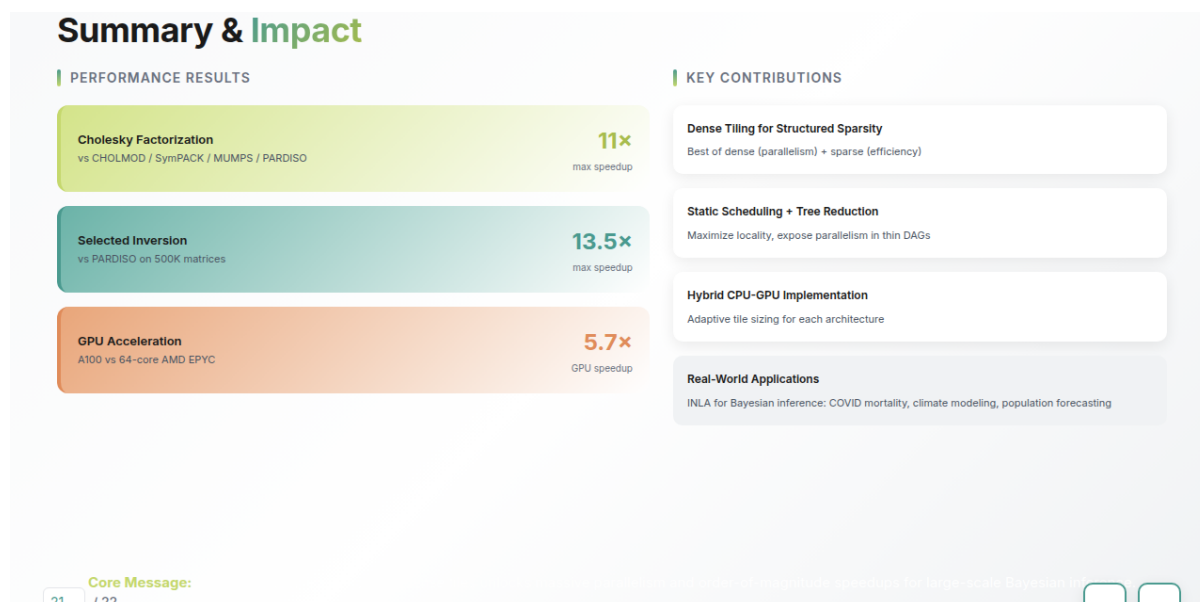
The first, on Cholesky factorization, was published at ISC High Performance 2025. It covers the sTiles framework, tile algorithms, ordering strategies, tree reduction, and CPU/GPU performance.

The second, on selected inversion, is targeting ISC 2026. It extends sTiles to compute inverse elements on the sparsity pattern, with the two-phase algorithm and additional GPU optimizations.

For future work, we're exploring multi-GPU support for even larger matrices, distributed memory implementations for cluster-scale problems, and integration with R-INLA for direct impact on statistical applications.

The software is open source, available on GitHub.

Slide 22: Summary



Speaker Notes

To summarize:

sTiles is a GPU-accelerated framework for sparse factorizations of structured matrices, particularly arrowhead patterns from Bayesian inference.

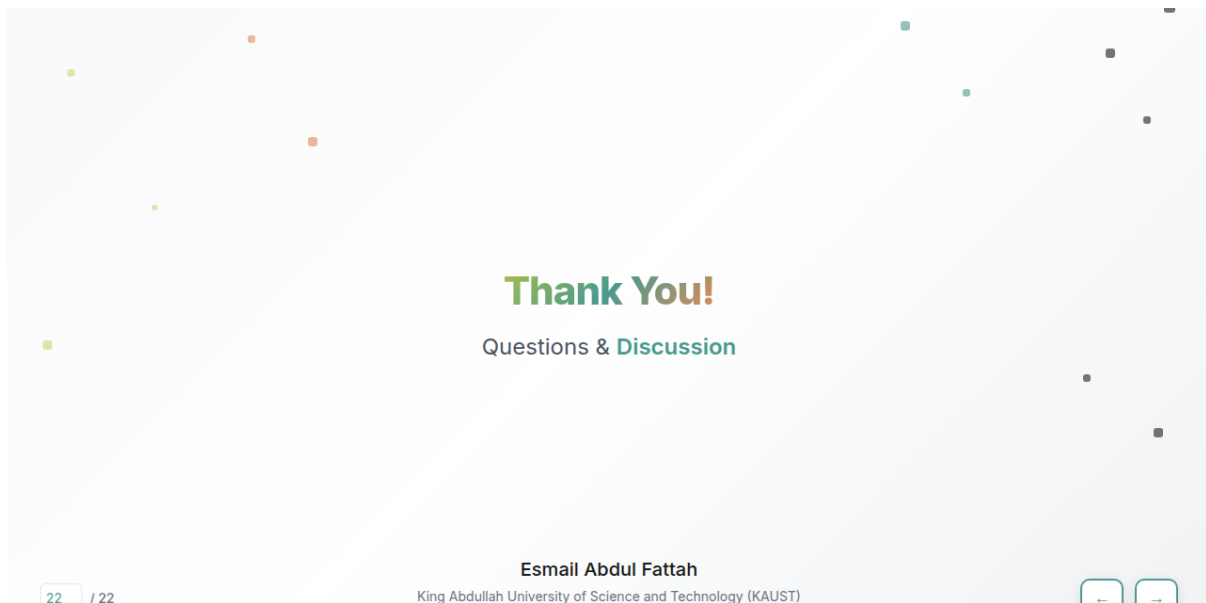
The key insight: combine sparse techniques — permutation, symbolic factorization, skipping zeros — with dense performance from tile-based BLAS-3 operations.

Results for Cholesky factorization: up to 11× speedup over PARDISO, with the gap growing at larger scales.

Results for selected inversion: up to 9.2× speedup over PARDISO on CPU, additional 5.7× speedup on GPU.

The practical impact: enabling faster Bayesian inference for large-scale spatial and spatiotemporal statistics, where these factorizations are often the computational bottleneck.

Slide 23: Thank You



Speaker Notes

Thank you for your attention.
The sTiles software is available at github.com/esmail-abdulfattah/sTiles.
I'm happy to take questions about the algorithms, implementation details, or results.

End of presentation script